

FAQ: Concurrency

MongoDB allows multiple clients to read and write the same data. To ensure consistency, MongoDB uses locking and concurrency control to prevent clients from modifying the same data simultaneously. Writes to a single document occur either in full or not at all, and clients always see consistent data.

What type of locking does MongoDB use?

MongoDB uses multi-granularity locking [1] that allows operations to lock at the global, database or collection level, and allows for individual storage engines to implement their own concurrency control below the collection level (e.g., at the document-level in WiredTiger).

MongoDB uses reader-writer locks that allow concurrent readers shared access to a resource, such as a database or collection.

In addition to a shared (S) locking mode for reads and an exclusive (X) locking mode for write operations, intent shared (IS) and intent exclusive (IX) modes indicate an intent to read or write a resource using a finer granularity lock. When locking at a certain granularity, all higher levels are locked using an intent lock.

For example, when locking a collection for writing (using mode X), both the corresponding database lock and the global lock must be locked in intent exclusive (IX) mode. A single database can simultaneously be locked in IS and IX mode, but an exclusive (X) lock cannot coexist with any other modes, and a shared (S) lock can only coexist with intent shared (IS) locks.

Locks are fair, with lock requests for reads and writes queued in order. However, to optimize throughput, when one lock request is granted, all other compatible lock requests are granted at the same time, potentially releasing the locks before a conflicting lock request is performed. For example, consider a situation where an X lock was just released and the conflict queue contains these locks:

IS → IS → X → X → S → IS

In strict first-in, first-out (FIFO) ordering, only the first two IS modes would be granted. Instead MongoDB will actually grant all IS and S modes, and once they all drain, it will grant X, even if new IS or S requests have been queued in the meantime. As a grant will always move all other requests ahead in the queue, no starvation of any request is possible.

In `db.serverStatus()` and `db.currentOp()` output, the lock modes are represented as follows:

Lock Mode	Description
R	Represents Shared (S) lock.
W	Represents Exclusive (X) lock.

Lock Mode	Description
<code>r</code>	Represents Intent Shared (IS) lock.
<code>w</code>	Represents Intent Exclusive (IX) lock.

[\[1\]](#) See the Wikipedia page on Multiple granularity locking [↗](#) for more information.

How granular are locks in MongoDB?

For most read and write operations, WiredTiger uses optimistic concurrency control. WiredTiger uses only intent locks at the global, database and collection levels. When the storage engine detects conflicts between two operations, one will incur a write conflict causing MongoDB to transparently retry that operation.

Some global operations, typically short lived operations involving multiple databases, still require a global "instance-wide" lock. Some other operations, such as `renameCollection`, still require an exclusive database lock in certain circumstances.

How do I see the status of locks on my `mongod` instances?

For reporting on lock utilization information on locks, use any of these methods:

- `db.serverStatus()`,
- `db.currentOp()`,
- `mongotop`,
- `mongostat`, and/or
- the MongoDB Cloud Manager or Ops Manager, an on-premise solution available in MongoDB Enterprise Advanced

Specifically, the `locks` document in the output of `serverStatus`, or the `locks` field in the `current operation reporting` provides insight into the type of locks and amount of lock contention in your `mongod` instance.

In `db.serverStatus()` and `db.currentOp()` output, the lock modes are represented as follows:

Lock Mode	Description
<code>R</code>	Represents Shared (S) lock.
<code>W</code>	Represents Exclusive (X) lock.
<code>r</code>	Represents Intent Shared (IS) lock.
<code>w</code>	Represents Intent Exclusive (IX) lock.

To terminate an operation, use `db.killOp()`.

Does a read or write operation ever yield the lock?

In some situations, read and write operations can yield their locks.

Long running read and write operations, such as queries, updates, and deletes, yield locks under many conditions. MongoDB operations can also yield locks between individual document modifications in write operations that affect multiple documents.

For storage engines supporting document level concurrency control, such as WiredTiger, yielding is not necessary when accessing storage as the intent locks, held at the global, database and collection level, do not block other readers and writers. However, operations will periodically yield, such as:

- to avoid long-lived storage transactions because these can potentially require holding a large amount of data in memory;
- to serve as interruption points so that you can kill long running operations;
- to allow operations that require exclusive access to a collection such as index/collection drops and creations.

What locks are taken by some common client operations?

The following table lists some operations and the types of locks they use for document level locking storage engines:

Operation	Database	Collection
Issue a query	 (Intent Shared)	 (Intent Shared)
Insert data	 (Intent Exclusive)	 (Intent Exclusive)
Remove data	 (Intent Exclusive)	 (Intent Exclusive)
Update data	 (Intent Exclusive)	 (Intent Exclusive)
Perform Aggregation	 (Intent Shared)	 (Intent Shared)
Create an index		 (Exclusive)
List collections	 (Intent Shared)	
Map-reduce	 (Exclusive) and  (Shared)	 (Intent Exclusive) and  (Intent Shared)

NOTE

Creating an index requires an exclusive (W) lock on a collection. However, the lock is not retained for the full duration of the index build process.

For more information, see [Index Builds on Populated Collections](#).

Which administrative commands lock a database?

Some administrative commands can exclusively lock a database for extended time periods. For large clusters, consider taking the `mongod` instance offline so that clients are not affected. For example, if a `mongod` is part of a replica set, take the `mongod` offline and let other members of the replica set process requests while maintenance is performed.

Administrative Commands Taking Extended Locks

These administrative operations require an exclusive lock at the database level for extended periods:

- `cloneCollectionAsCapped` command
- `convertToCapped` command

In addition, the `renameCollection` command and corresponding `db.collection.renameCollection()` shell method take the following locks:

Command	Lock behavior
<code>renameCollection</code> database command	If renaming a collection within the same database, the <code>renameCollection</code> command takes an exclusive (W) lock on the source and target collections. If the target namespace is in a different database as the source collection, The <code>renameCollection</code> command takes an exclusive (W) lock on the target database when renaming a collection across databases and blocks other operations on that database until it finishes.
<code>renameCollection()</code> shell helper method	The <code>renameCollection()</code> method takes an exclusive (W) lock on the source and target collections, and cannot move a collection between databases.

Which administrative commands lock a collection?

These administrative operations require an exclusive lock at the collection level:

- `create` command and corresponding `db.createCollection()` and `db.createView()` shell methods
- `createIndexes` command and corresponding `db.collection.createIndex()` and `db.collection.createIndexes()` shell methods
- `drop` command and corresponding `db.collection.drop()` shell methods
- `dropIndexes` command and corresponding `db.collection.dropIndex()` and `db.collection.dropIndexes()` shell methods

- the `renameCollection` command and corresponding `db.collection.renameCollection()` shell method take the following locks, depending on version:
 - For `renameCollection` and `db.collection.renameCollection()`: If renaming a collection within the same database, the operation takes an exclusive (W) lock on the source and target collections.
 - For `renameCollection` *only*: If the target namespace is in a different database as the source collection, the operation takes an exclusive (W) lock on the target database when renaming a collection across databases and blocks other operations on that database until it finishes.
- the `reIndex` command and corresponding `db.collection.reIndex()` shell method obtain an exclusive (W) lock on the collection and block other operations on the collection until finished.
- the `replSetResizeOplog` command takes an exclusive (W) lock on the `oplog` collection and blocks other operations on the collection until it finishes.

Does a MongoDB operation ever lock more than one database?

These MongoDB operations may obtain and hold a lock on more than one database:

Operation	Behavior
<code>reIndex</code> <code>db.collection.reIndex()</code>	These operations only obtain an exclusive (W) collection lock instead of a global exclusive lock.
<code>renameCollection</code>	This operation obtains an exclusive (W) lock on the target database, an intent shared (r) lock on the source database, and a shared (S) lock on the source collection when renaming a collection across databases. When renaming a collection in the same database, the operation only requires exclusive (W) locks on the source and target collections.
<code>replSetResizeOplog</code>	This operation only obtains an exclusive (W) lock on the <code>oplog</code> collection instead of a global exclusive lock.

How does sharding affect concurrency?

Sharding improves concurrency by distributing collections over multiple `mongod` instances, allowing shard servers (specifically, `mongos` processes) to run concurrently with the downstream `mongod` instances.

In a sharded cluster, locks apply to each individual shard, not to the whole cluster; i.e.

each `mongod` instance is independent of the others in the sharded cluster and uses its own locks. The operations on one `mongod` instance do not block the operations on any others.

How does concurrency affect a replica set primary?

With replica sets, when MongoDB writes to a collection on the primary, MongoDB also writes to the primary's oplog, which is a special collection in the `local` database. Therefore, MongoDB must lock both the collection's database and the `local` database. The `mongod` must lock both databases at the same time to keep the database consistent and ensure that write operations, even with replication, are all or nothing operations.

When writing to a replica set, the lock's scope applies to the primary.

How does concurrency affect secondaries?

In replication, MongoDB does not apply writes serially to secondaries. Secondaries collect oplog entries in batches and then apply those batches in parallel. Writes are applied in the order that they appear in the oplog.

Reads that target secondaries read from a WiredTiger snapshot of the data if the secondary is undergoing replication. This allows the read to occur simultaneously with replication, while still guaranteeing a consistent view of the data.

Does MongoDB support transactions?

Because a single document can contain related data that would otherwise be modeled across separate parent-child tables in a relational schema, MongoDB's atomic single-document operations already provide transaction semantics that meet the data integrity needs of the majority of applications. One or more fields may be written in a single operation, including updates to multiple sub-documents and elements of an array. The guarantees provided by MongoDB ensure complete isolation as a document is updated; any errors cause the operation to roll back so that clients receive a consistent view of the document.

For situations that require atomicity of reads and writes to multiple documents (in a single or multiple collections), MongoDB supports distributed transactions, including transactions on replica sets and sharded clusters.

For more information, see [transactions](#).

IMPORTANT

In most cases, a distributed transaction incurs a greater performance cost over single document writes, and the availability of distributed transactions should not be a replacement for effective schema design. For many scenarios, the denormalized data model (embedded documents and arrays) will continue to be optimal for your data and use cases. That is, for many scenarios, modeling your data appropriately will minimize the need for distributed transactions.

For additional transactions usage considerations (such as runtime limit and oplog size limit), see also [Production Considerations](#).

What isolation guarantees does MongoDB provide?

Depending on the read concern, clients can see the results of writes before the writes are durable. To control whether the data read may be rolled back or not, clients can use the `readConcern` option.

What are lock-free read operations?

New in version 5.0.

A lock-free read operation runs immediately: it is not blocked when another operation has an exclusive (X) write lock on the collection.

Starting in MongoDB 5.0, the following read operations are not blocked when another operation holds an exclusive (X) write lock on the collection:

- `find`
- `count`
- `distinct`
- `aggregate`
- `mapReduce`
- `listCollections`
- `listIndexes`

When writing to a collection, `mapReduce` and `aggregate` hold an intent exclusive (IX) lock. Therefore, if an exclusive X lock is already held on a collection, `mapReduce` and `aggregate` write operations are blocked.

For information, see:

- Read Isolation, Consistency, and Recency
- Atomicity and Transactions
- Read Concern